Software Engineering I (02161)

Testing and Test Automation

Assoc. Prof. Hubert Baumeister

DTU Compute
Technical University of Denmark

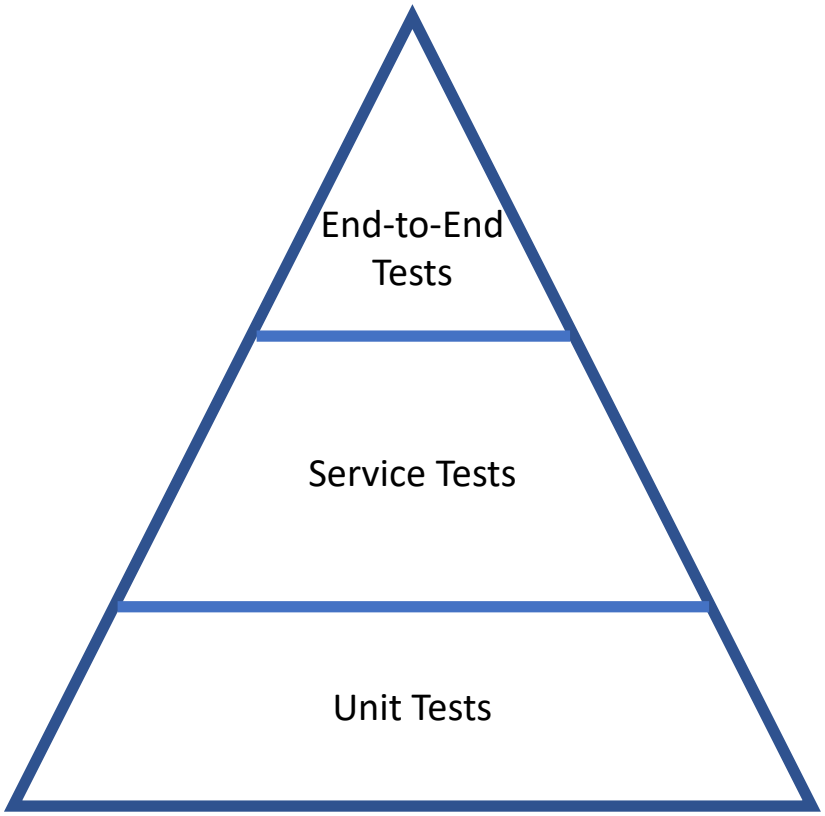
Purpose of tests

- Find bugs
- Give evidence that the software works as intended
  - Caveat: Green tests does not mean the software is correct (Edsgar Dijkstra)
  - → All requirements have tests
  - → All code is covered by tests

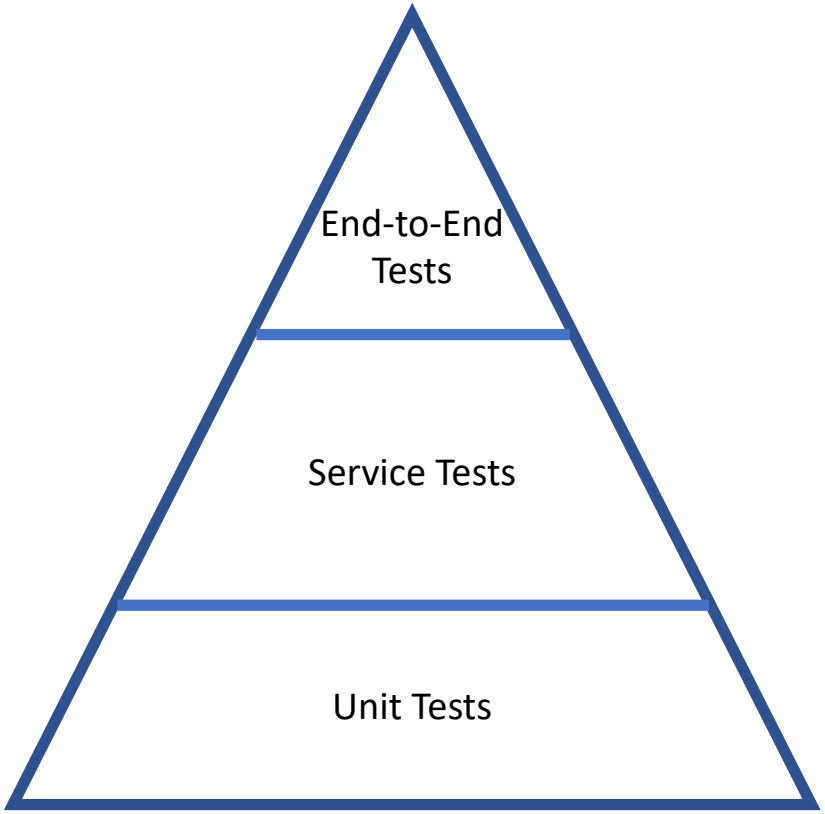
Types of Tests

	Supports Programming	Critique Project
Business Facing	**Acceptance Tests**	**Explorative Testing (trying to break the system)**
Technology Facing	**Unit Tests**	**Property Testing (Performance, Scalability)**

Test Pyramid



Test Pyramid



End-to-End tests (Acceptance tests)

- Tests the whole application
- → Can be very expensive
- → Test under the UI

Service Tests

- Tests Microservices by stubbing all dependent services
- → Faster, in particular when not going through the adaptors
- → Hexagonal architecture

Unit Tests

- → Usually very fast

Example of an end-to-end test

- Use case

  name: Login Admin
  actor: Admin
  precondition: Admin is not logged in
  main scenario

  1. Admin enters password
  2. System responds true

  alternative scenarios:

  1a. Admin enters wrong password
  1b. The system reports that the password is wrong and the use case starts from the beginning

  postcondition: Admin is logged in

Manual tests

Successful login

Prerequisit: the password for the administrator is “adminadmin”

Input	Step	Expected Output	Fail	OK
	Startup system	“0) Exit” “1) Login as administrator”		
“1”	Enter choice	“password”		
“adminadmin”	Enter string	“logged in”		

Failed login

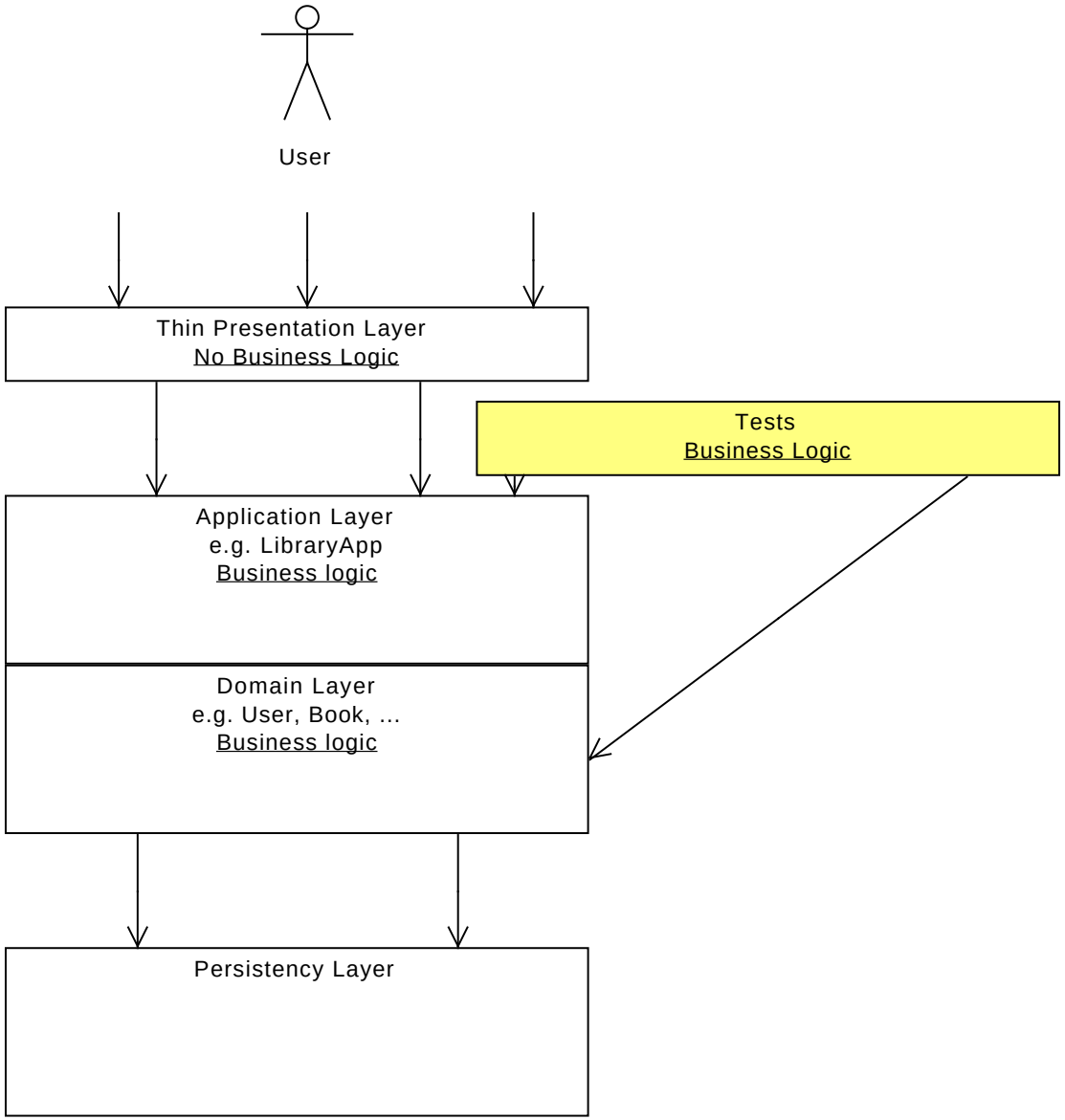
Prerequisit: the password for the administrator is “adminadmin”

Input	Step	Expected Output	Fail	OK
	Startup system	“0) Exit” “1) Login as administrator”		
“1”	Enter choice	“password”		
“admin”	Enter string	“Password incorrect” “0) Exit” “1) Login as administrator”		

Manual vs. automated tests?

- Manual tests
  - Ad hoc → explorative testing
  - → need planning
  - Are expensive; can't be run often
  - → should be reduced to a minimum
- Automated tests
  - Are cheap; can be run often
  - Help automate deployment
- Robert Martin (Uncle Bob) in `https://youtu.be/hG4LH6P8Syk`
  - manual tests are immoral from 36:35 (`https://youtu.be/hG4LH6P8Syk?t=2195`)
  - how to test applications having a UI from 40:00 (`https://youtu.be/hG4LH6P8Syk?t=2400`)
- How to do UI tests?
  - → Solution: Test under the UI

Test under the UI



Automated test using JUnit

Successful login

```
@Test
public void testLoginAdmin() {
   LibraryApp libApp = new LibraryApp();

   assertFalse(libApp.adminLoggedIn());

   boolean login = libApp.adminLogin("adminadmin");

   assertTrue(login);
   assertTrue(libApp.adminLoggedIn());
}
```

Failed login

```
@Test
public void testWrongPassword() {
   LibraryApp libApp = new LibraryApp();

   assertFalse(libApp.adminLoggedIn());

   boolean login = libApp.adminLogin("admin");

   assertFalse(login);
   assertFalse(libApp.adminLoggedIn());
}
```

Automated test as a Cucumber Feature

```
Feature: Admin login
   Description: The administrator logs into the library system
   Actor: Administrator

Scenario: Administrator can login
   Given that the administrator is not logged in
   And the password is "adminadmin"
   When the administrator logs in
   Then the adminstrator is logged in

Scenario: Administrator has the wrong password
   Given that the administrator is not logged in
   And the password is "wrong password"
   When the administrator logs in
   Then the login fails
```

Step definitions (excerpt)

```
@Given("^the password is \"([^\"]*)\"$")
public void thePasswordIs(String password) throws Exception {
   this.password = password;
}

@When("^the administrator logs in$")
public void theAdministratorLogsIn() throws Exception {
   libraryApp.adminLogin(password);
}
```